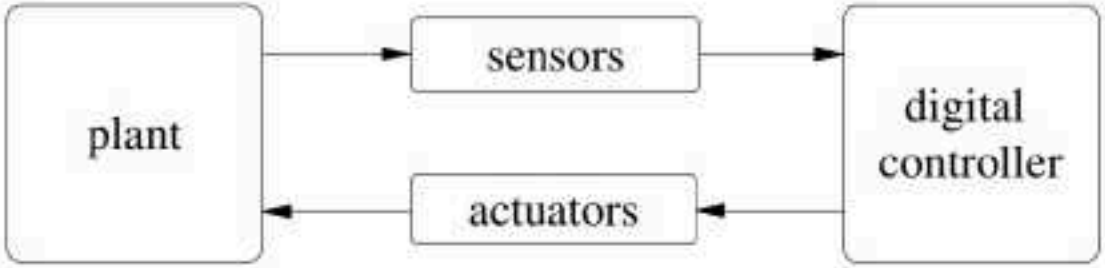
Outline

- Autonomous Systems and Their Correctness Verification (Introduction)
- Formal Methods Practice: Recently developed Tool framework
- Formal Methods Theory(as Tools of the Trade)
  - Model Checking
  - Static Analysis
  - Abstract Interpretation
- Open Challenges

Employed in diverse areas such as . . .



- Enable automation: simple ON/OFF and/or sophisticated control
- New incarnations: CPS, IoT, Mechatronics, etc.
- Exciting incarnations: Autonomous systems, robotic systems, drones, UAVs, AUVs, etc.
- Uniqueness:
  - Real-time
  - Non-terminating
  - Infinite stated (cont. dynamics)
  - Hybrid
- Most importantly, majority of them are SAFETY CRITICAL

Embedded Systems (safety critical) are everywhere (HIL)

Safety criticality mandates established correctness

Safety critical systems

Their failure might have catastrophic consequences, such as:

- Loss of life and
- Huge financial implications

- We need to have high confidence in their correctness
- Evidence-for that they behave correctly (as expected)
- Solution: Formal Methods

Autonomous Systems: What it means?

Autonomous Control Systems

Control systems with a power and ability for self-governance in the performance of control functions

- setting the goal/s and
- planning the course of actions to achieve the goal/s.

- Must perform well under significant uncertainities of the plant/environment
- For extended periods of time!!!!!
- involves controllers implementing High-level decision making techniques for reasoning under UC and to act accordingly
- requires intelligent autonomous controllers

Autonomous Systems include ...

DRONES / UAVS

Autonomous Under-Water Vehicles

MEDICAL ROBOTICS

HOME ROBOTICS

Autonomous Systems Software: Concerns?

Capabilities means Concerns!!!

- Autonomous systems rely on intelligent inference capabilities to be able to take appropriate actions even in unforeseen circumstances.
- It is typically very difficult to assess the reliability of autonomy software.

- For this reason, UAV/Drones are restricted to “Segregated” airspace
- Lack of confidence in their correctness
- Certification authorities would soon be commissioned
- Ex: RTCA (also EUROCAE) defined standards such as DO-178 178-B and 178-C
- Correctness, not absolute but as per specification!!!!!

Formal Verification (FV)

- applies mathematics to solve the system (HW/SW) correctness problem

Definition (System correctness problem)

R: requirement (what) D: design (how)

D conforms with R

- involves: formal specification and proof methods

Formal Verification (FV)

- applies mathematics to solve the system (HW/SW) correctness problem

Definition (System correctness problem)

R: requirement (what) D: design (how)

D conforms with R

- involves: formal specification and proof methods

Real-time system

Required property

Formal Verification (FV)

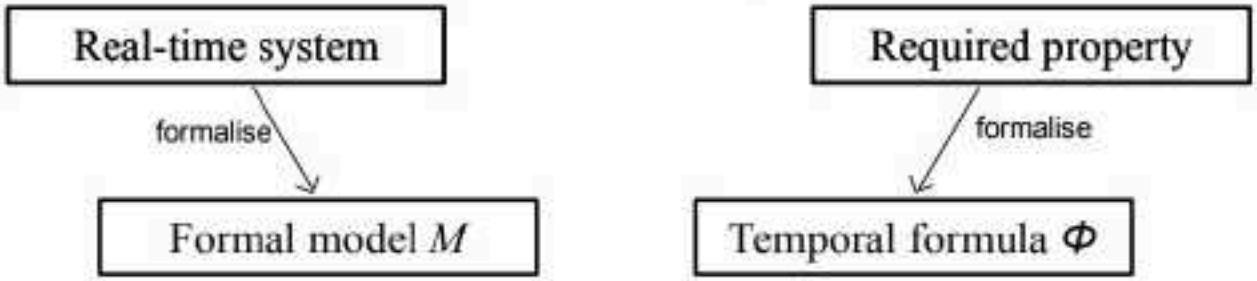
- applies mathematics to solve the system (HW/SW) correctness problem

Definition (System correctness problem)

R: requirement (what) D: design (how)

D conforms with R

- involves: formal specification and proof methods



Formal Verification (FV)

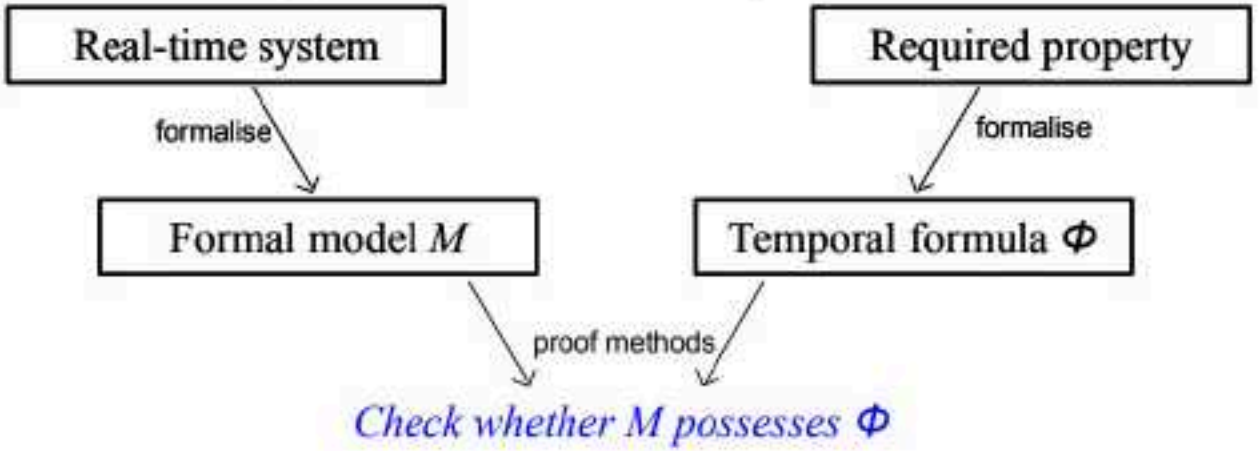
- applies mathematics to solve the system (HW/SW) correctness problem

Definition (System correctness problem)

R: requirement (what) D: design (how)

D conforms with R

- involves: formal specification and proof methods



Formal Verification (FV)

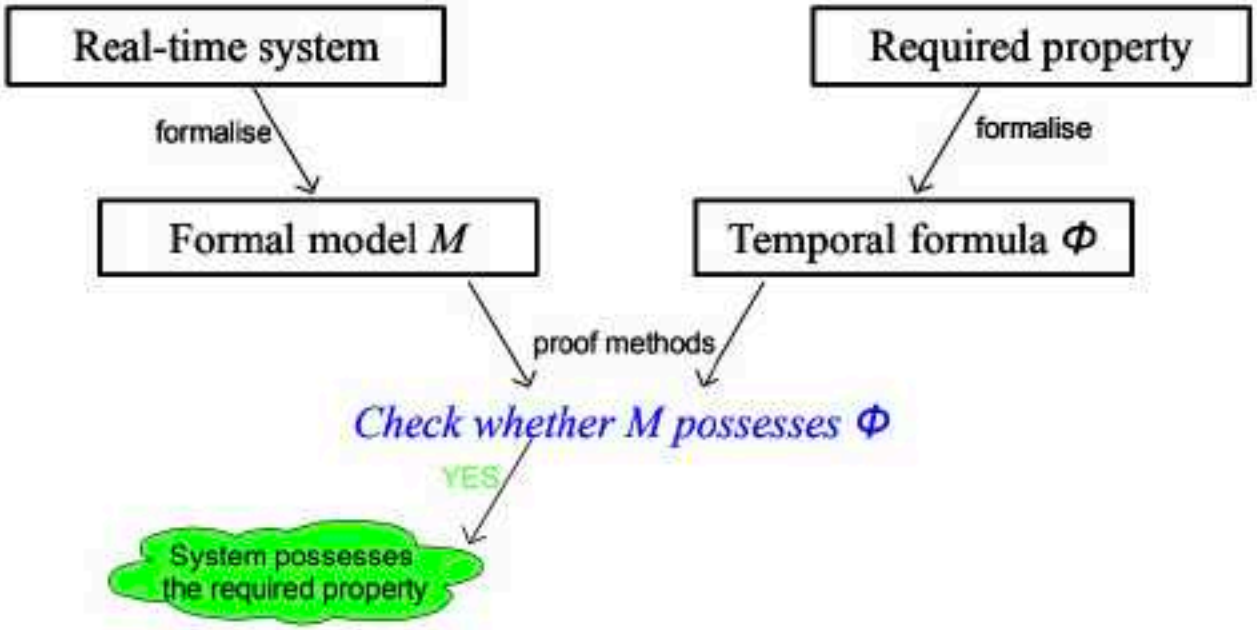
- applies mathematics to solve the system (HW/SW) correctness problem

Definition (System correctness problem)

R: requirement (what) D: design (how)

D conforms with R

- involves: formal specification and proof methods



Formal Verification (FV)

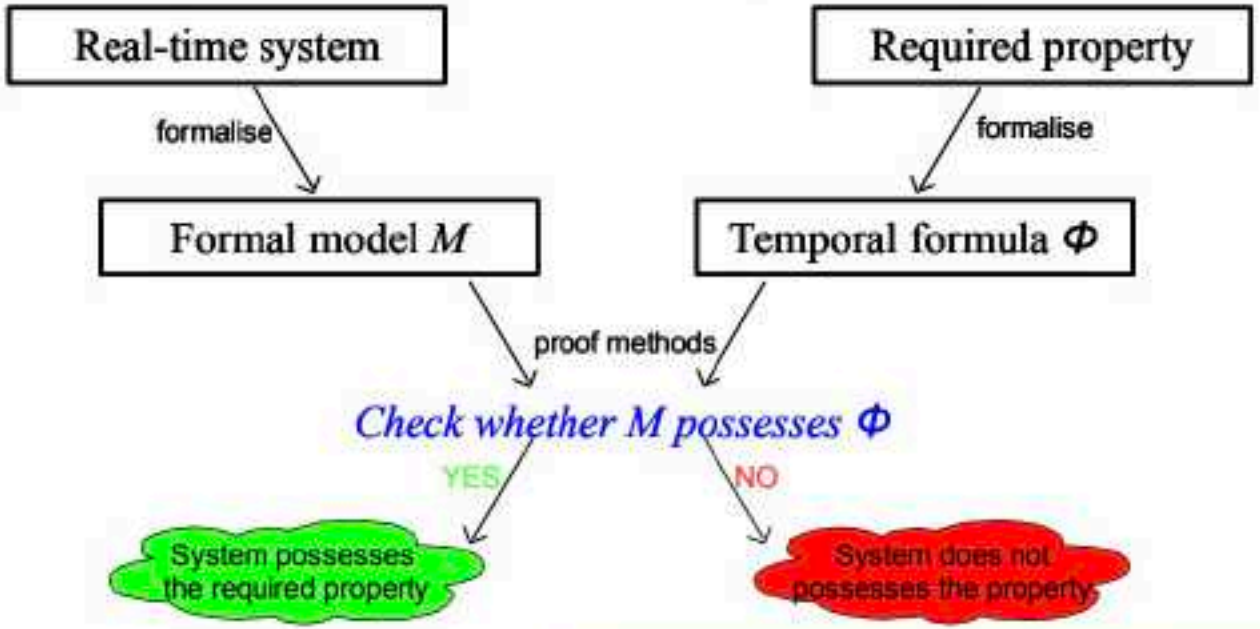
- applies mathematics to solve the system (HW/SW) correctness problem

Definition (System correctness problem)

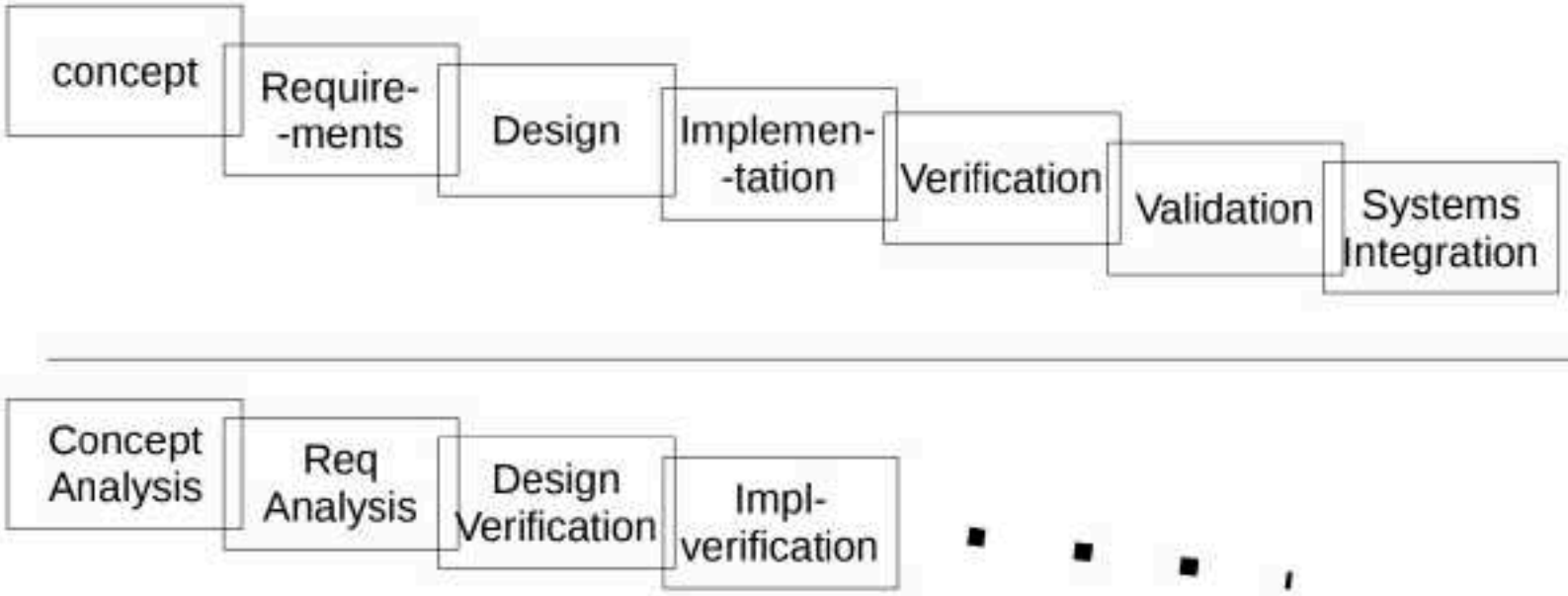
R: requirement (what) D: design (how)

D conforms with R

- involves: formal specification and proof methods

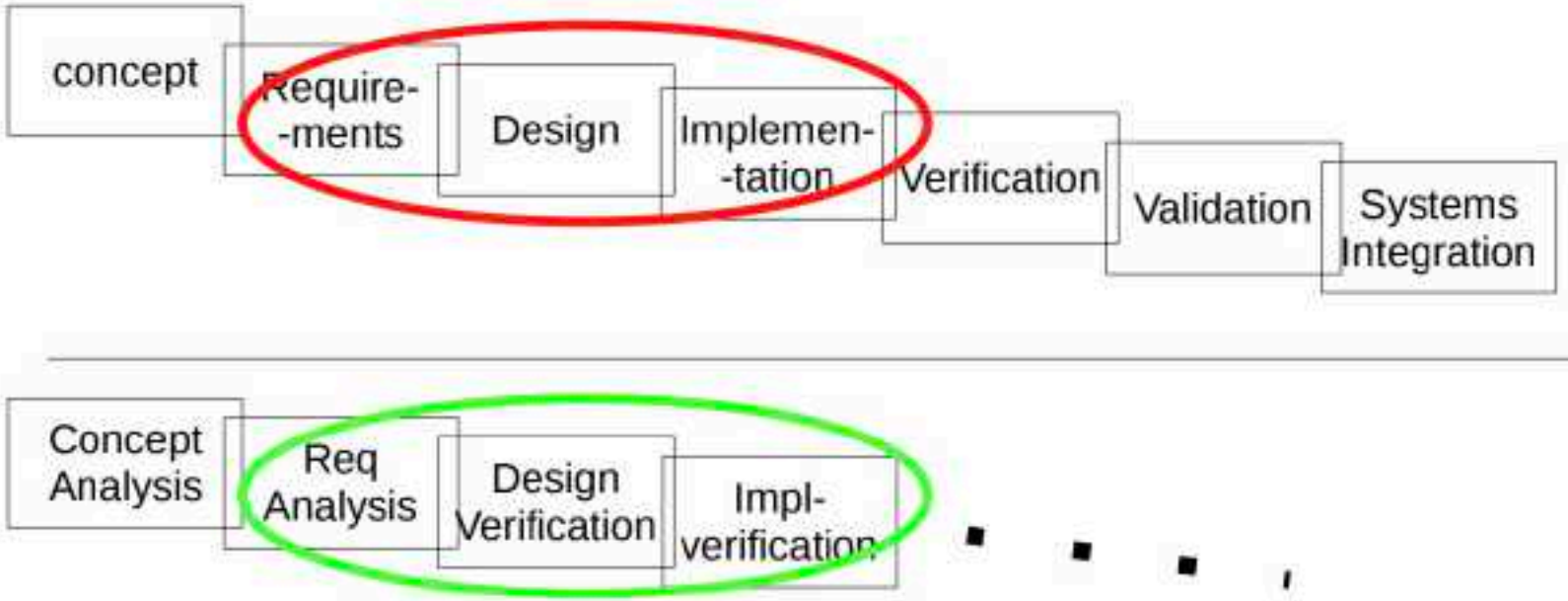


FV and SDLC



Several Formal Languages and Tools exist applicable in each of these phases

FV and SDLC



Theorem Provers: PVS, Coq, Isabella, etc.

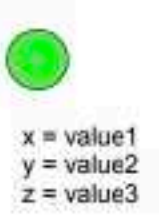
Model checkers: SPIN, Uppaal, NuSMV, CBMC, etc.

Static Analysers: Astree, Splint, Fluctuat, etc.

HYBRIDS

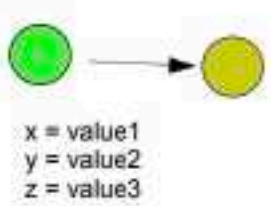
Naive approach to Correctness Analysis

Trace the states through which the system transits:



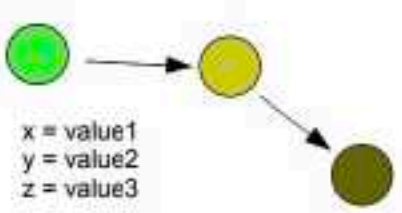
Naive approach to Correctness Analysis

Trace the states through which the system transits:



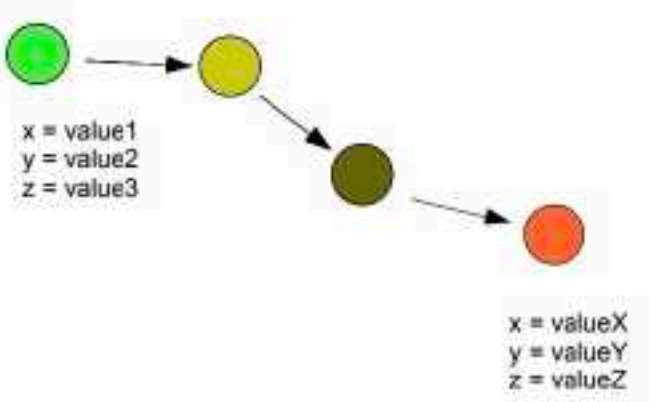
Naive approach to Correctness Analysis

Trace the states through which the system transits:



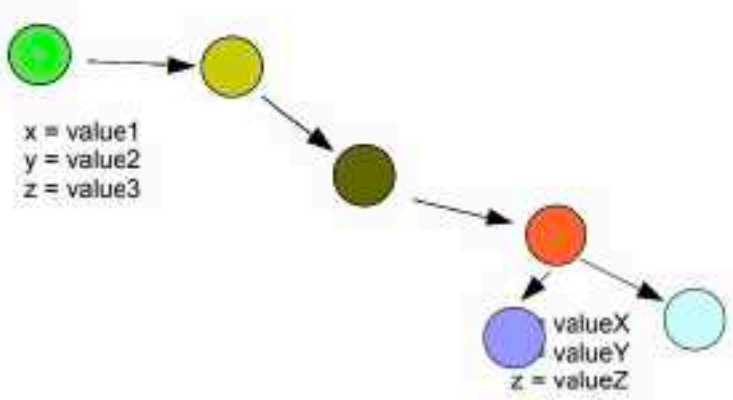
Naive approach to Correctness Analysis

Trace the states through which the system transits:



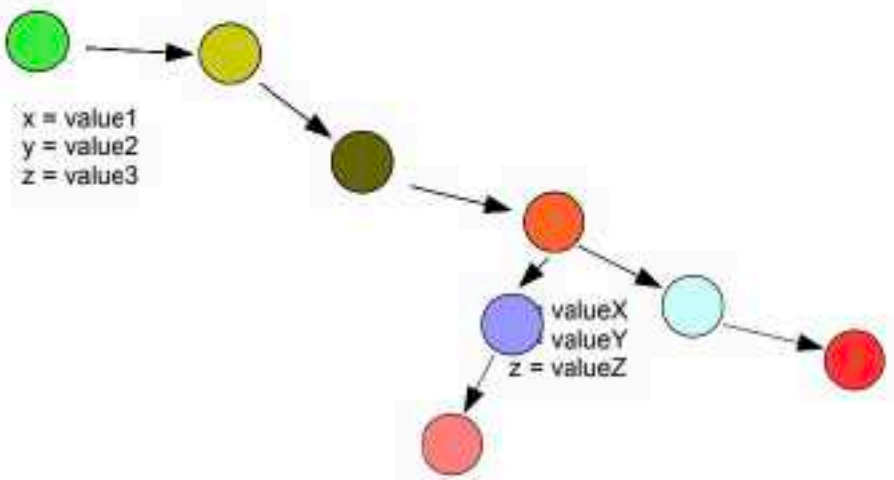
Naive approach to Correctness Analysis

Trace the states through which the system transits:



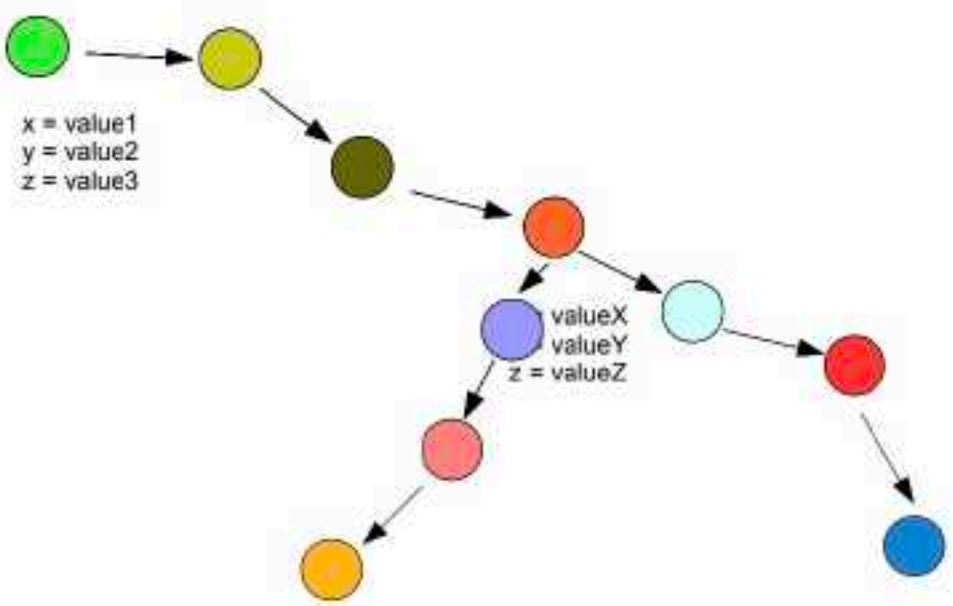
Naive approach to Correctness Analysis

Trace the states through which the system transits:



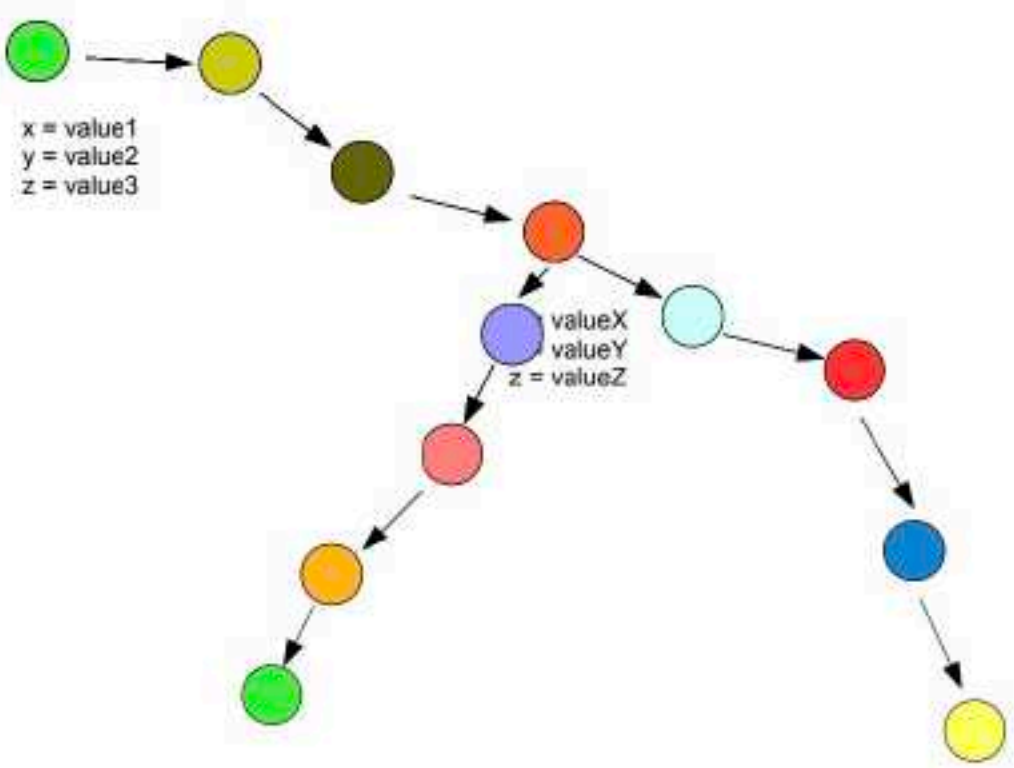
Naive approach to Correctness Analysis

Trace the states through which the system transits:



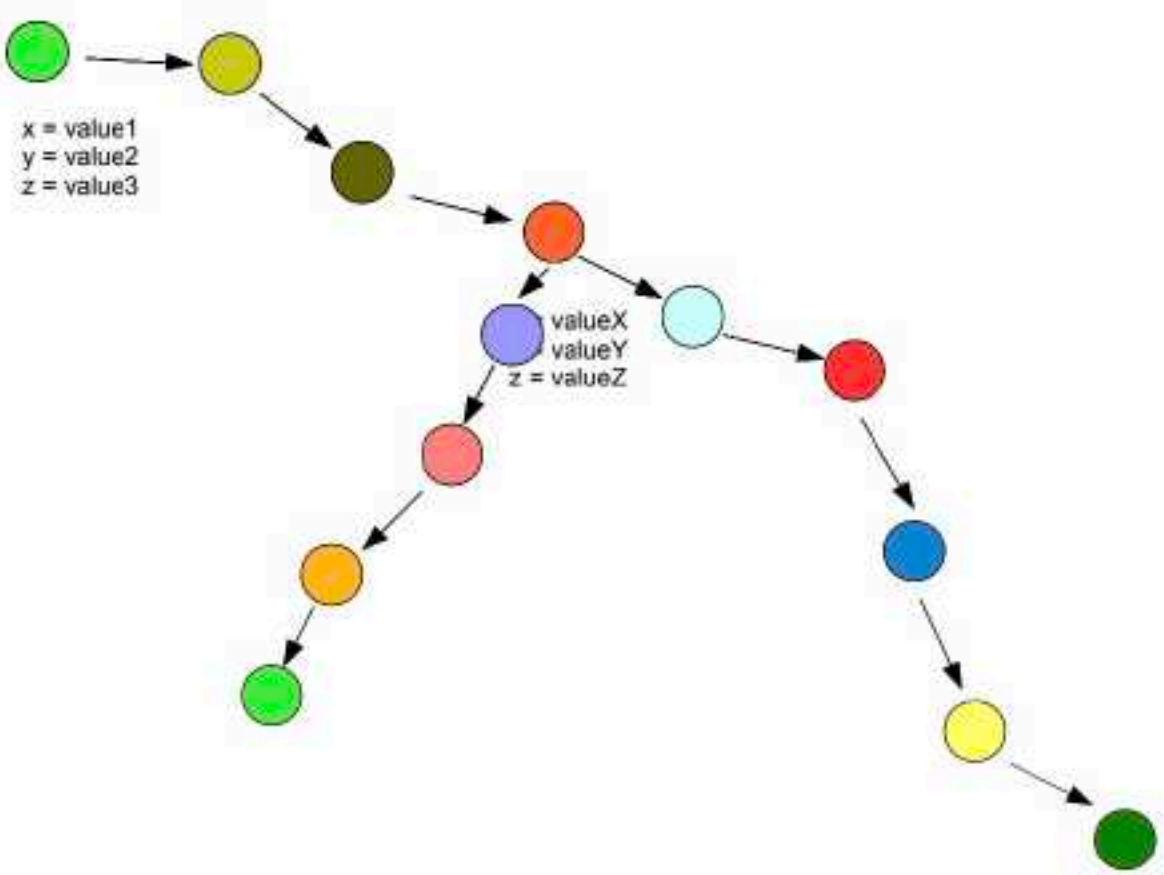
Naive approach to Correctness Analysis

Trace the states through which the system transits:



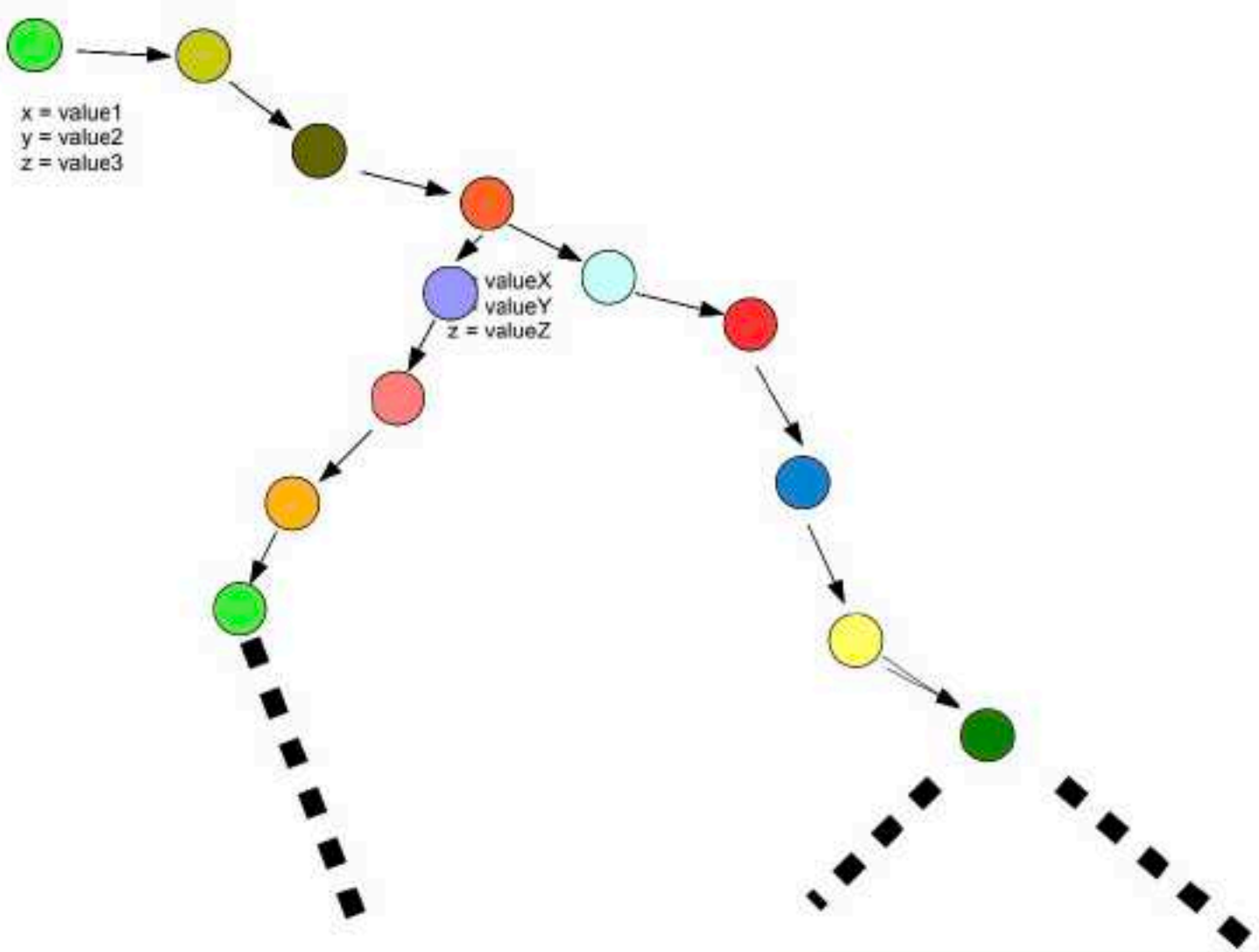
Naive approach to Correctness Analysis

Trace the states through which the system transits:



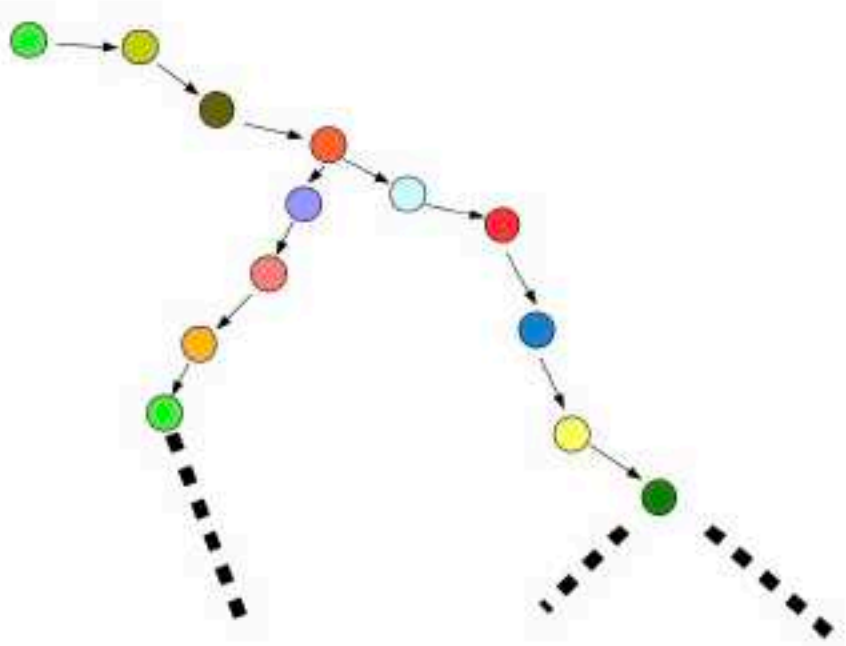
Naive approach to Correctness Analysis

Trace the states through which the system transits:



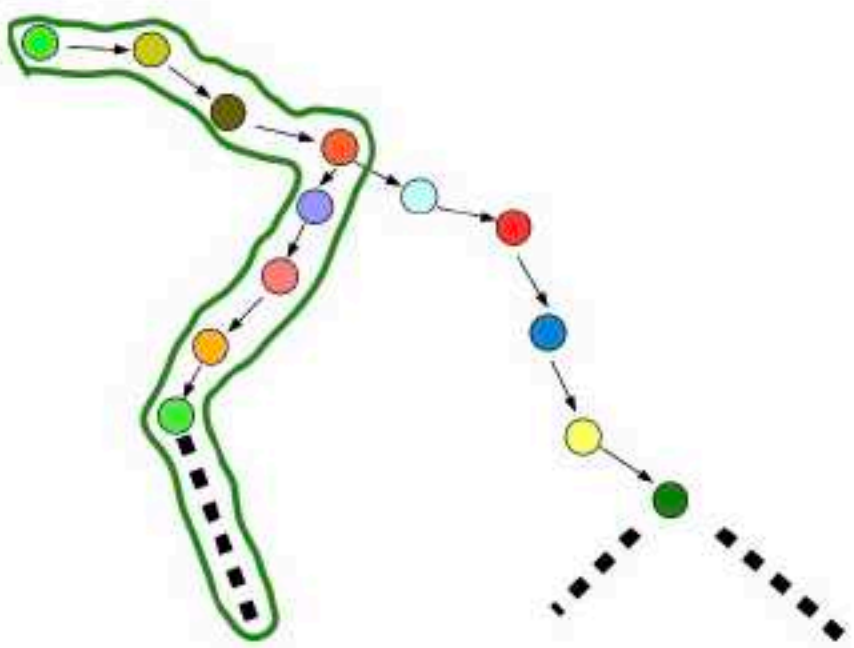
Multiple frontiers for execution

Compute all the distinct paths of system evolution frontiers:
(NOT ALWAYS POSSIBLE)



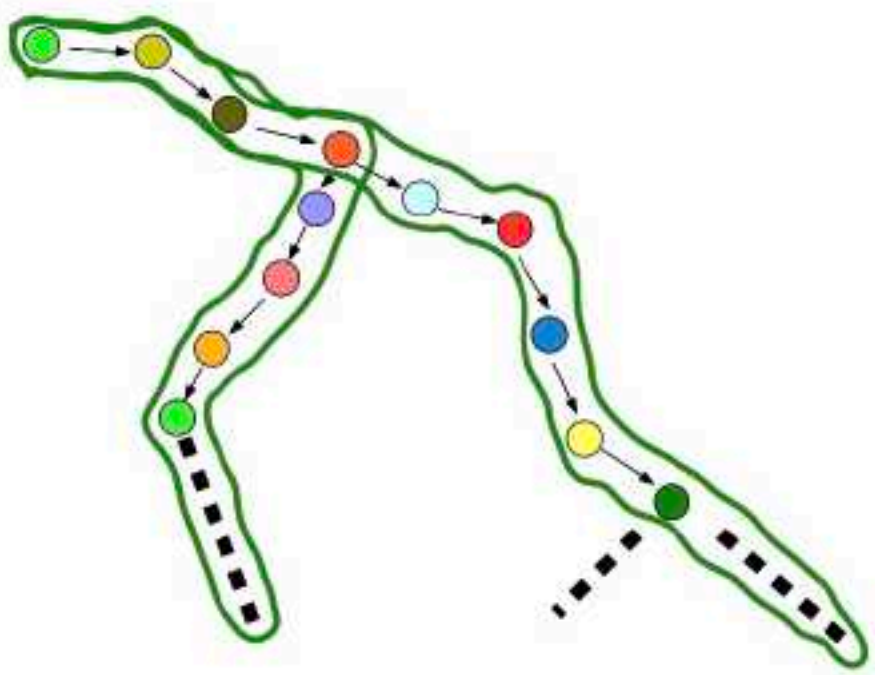
Multiple frontiers for execution

Compute all the distinct paths of system evolution frontiers:
(NOT ALWAYS POSSIBLE)



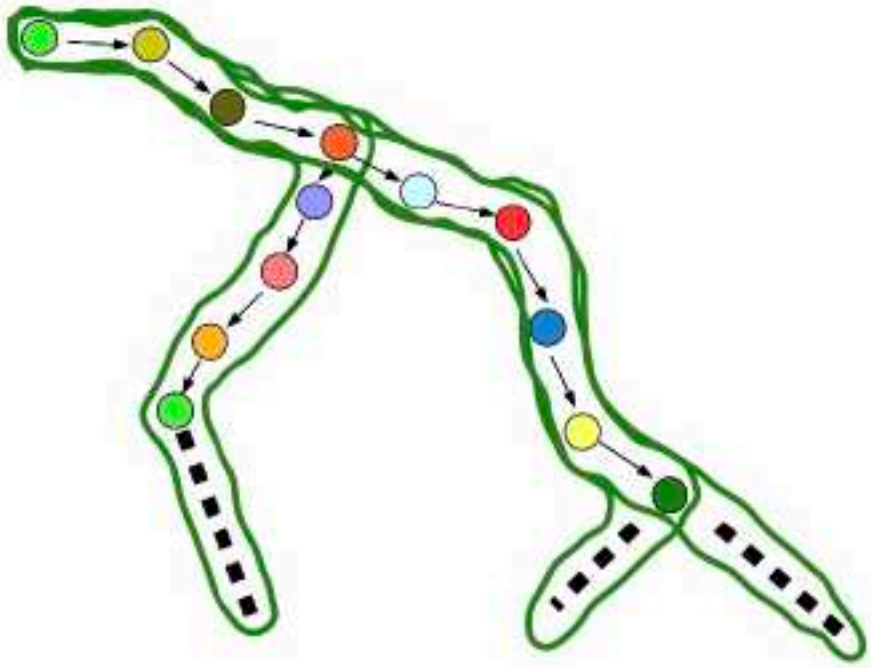
Multiple frontiers for execution

Compute all the distinct paths of system evolution frontiers:
(NOT ALWAYS POSSIBLE)



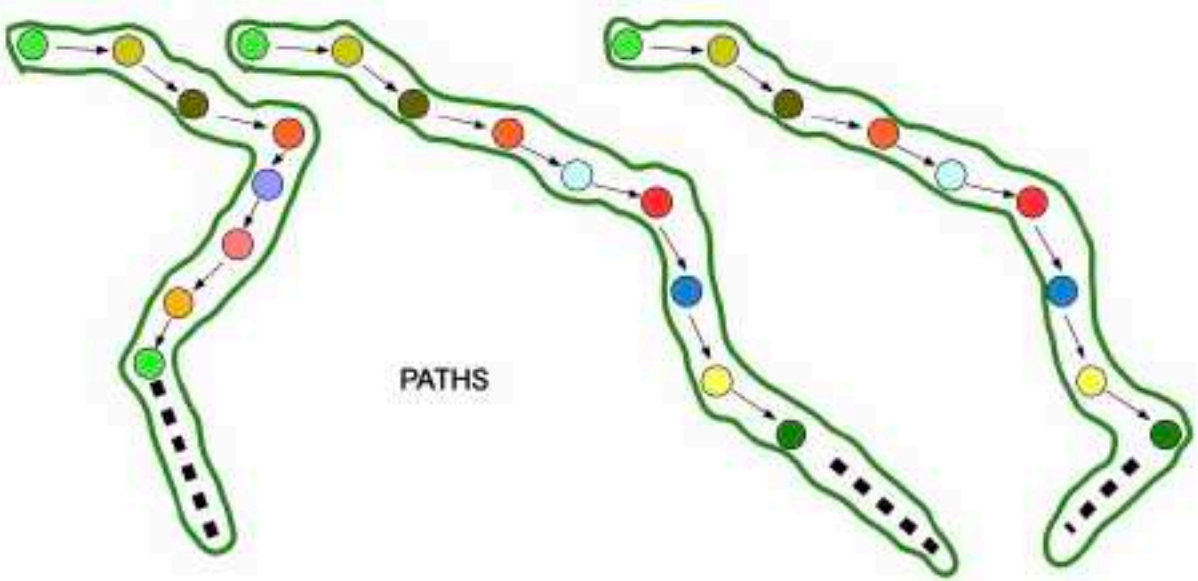
Multiple frontiers for execution

Compute all the distinct paths of system evolution frontiers:
(NOT ALWAYS POSSIBLE)



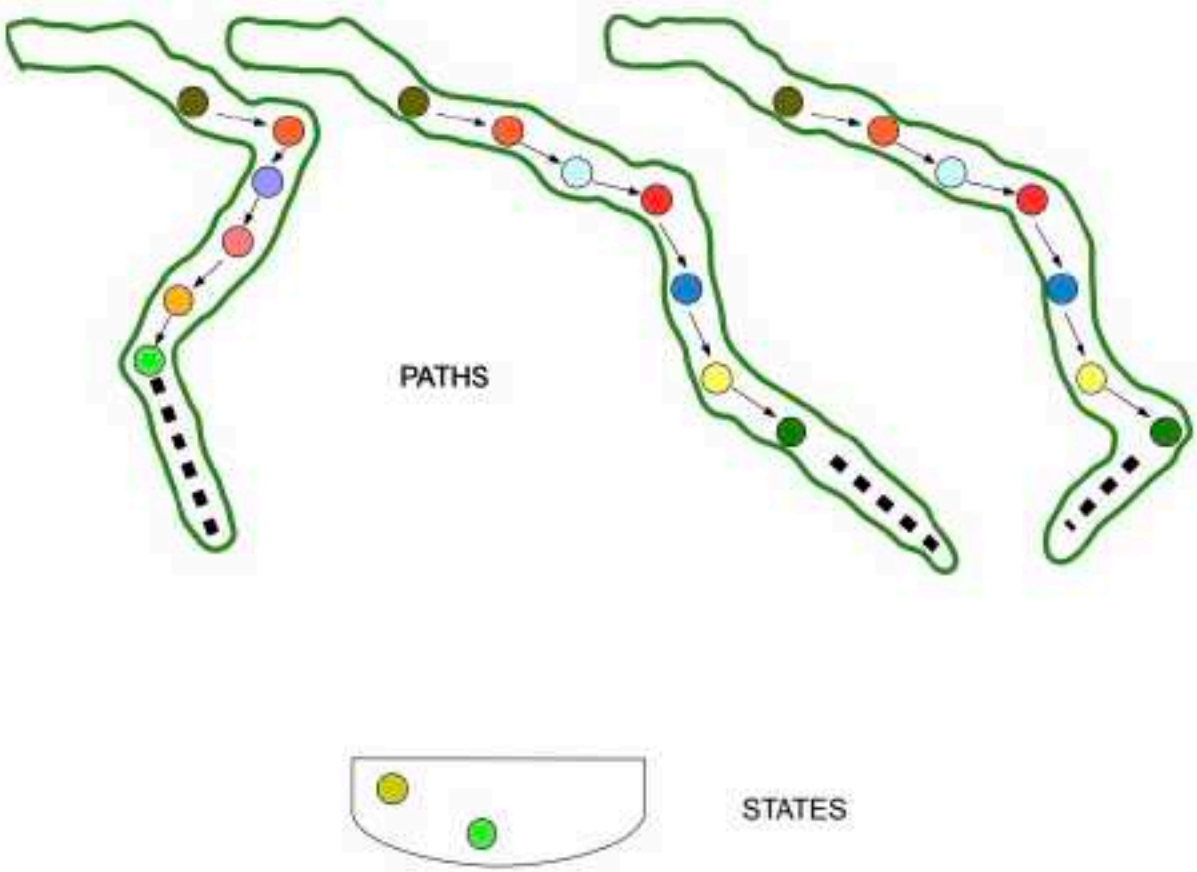
Collect the states (Reachable States)

Compute the states through which the system transits: (NOT ALWAYS POSSIBLE!!!!!)



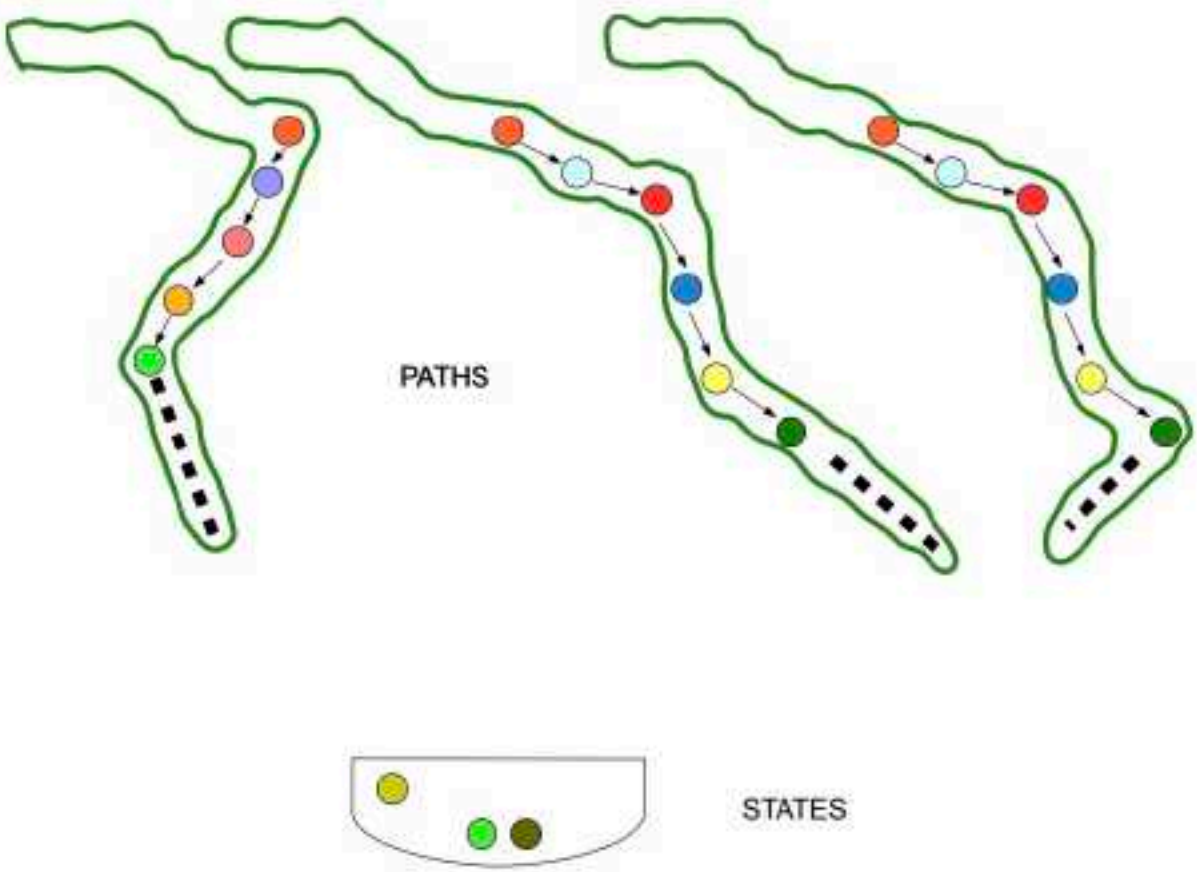
Collect the states (Reachable States)

Compute the states through which the system transits: (NOT ALWAYS POSSIBLE!!!!!)



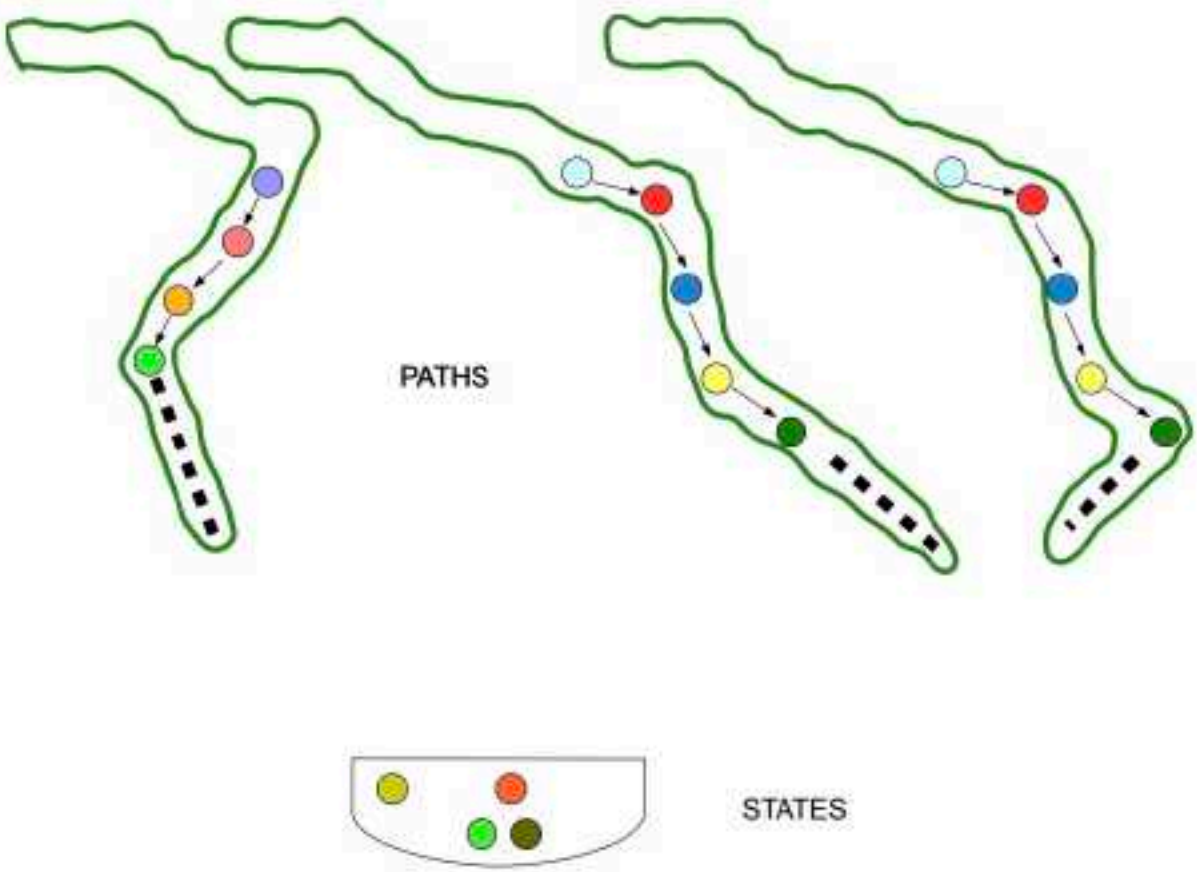
Collect the states (Reachable States)

Compute the states through which the system transits: (NOT ALWAYS POSSIBLE!!!!!)



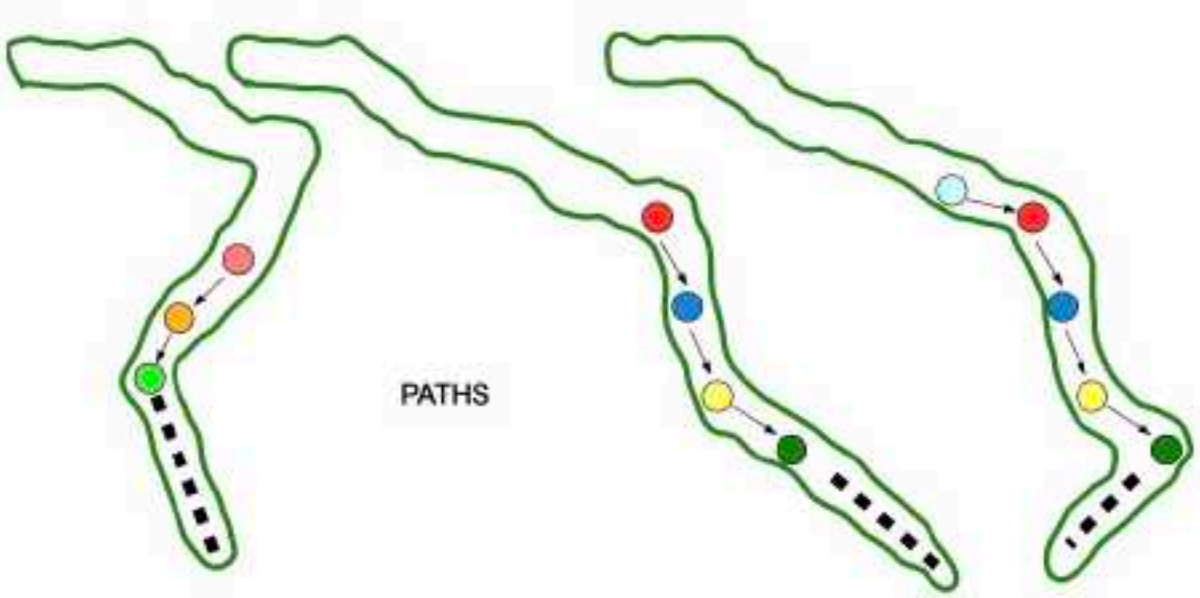
Collect the states (Reachable States)

Compute the states through which the system transits: (NOT ALWAYS POSSIBLE!!!!!)



Collect the states (Reachable States)

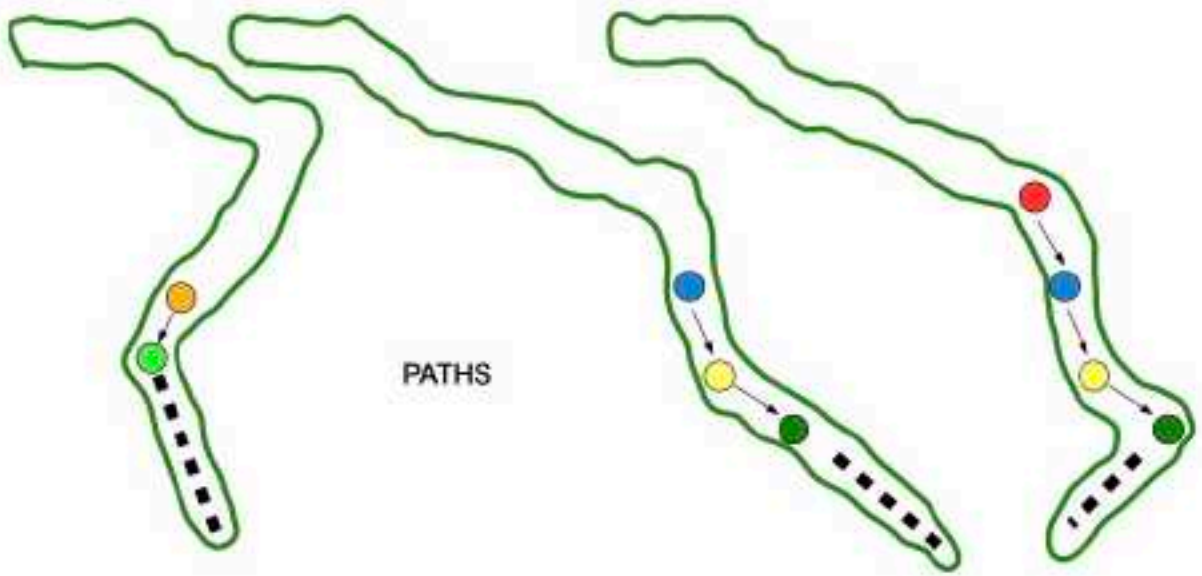
Compute the states through which the system transits: (NOT ALWAYS POSSIBLE!!!!!)



STATES

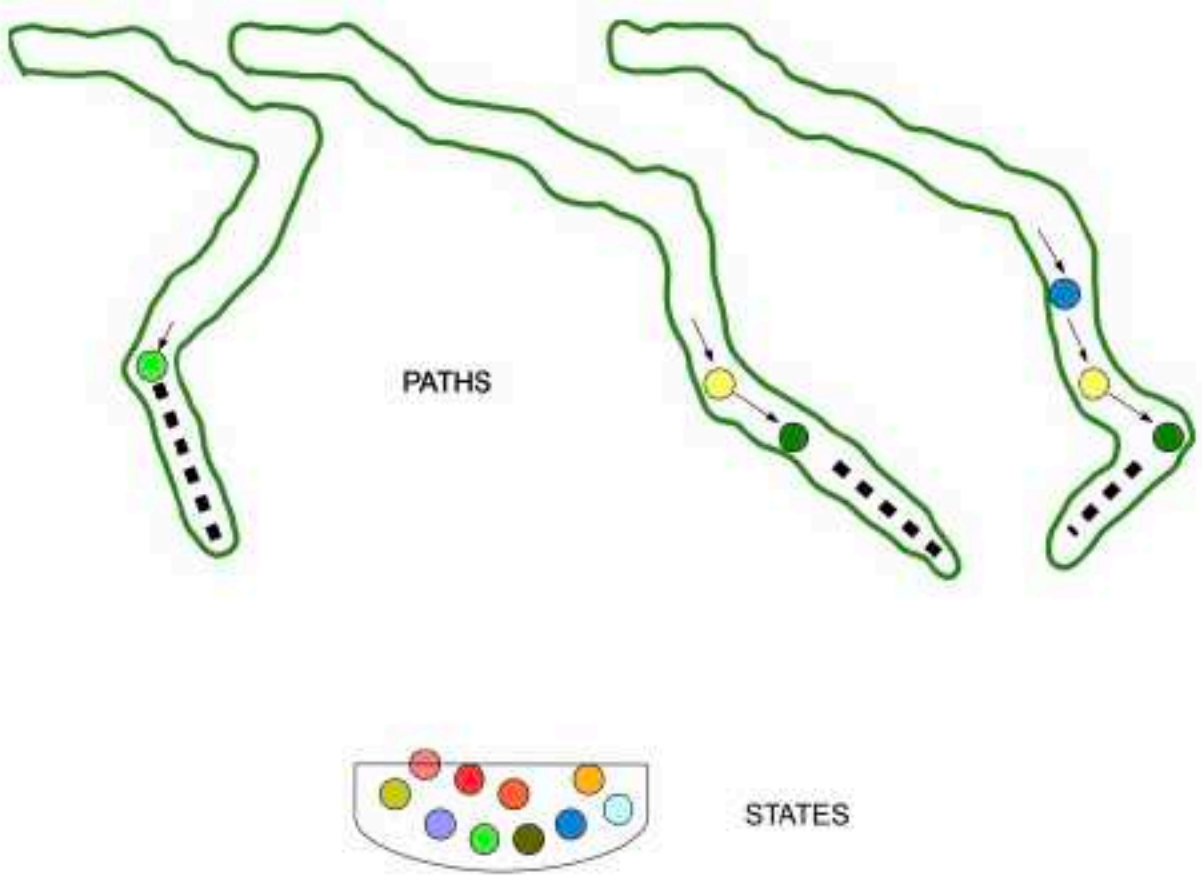
Collect the states (Reachable States)

Compute the states through which the system transits: (NOT ALWAYS POSSIBLE!!!!!)



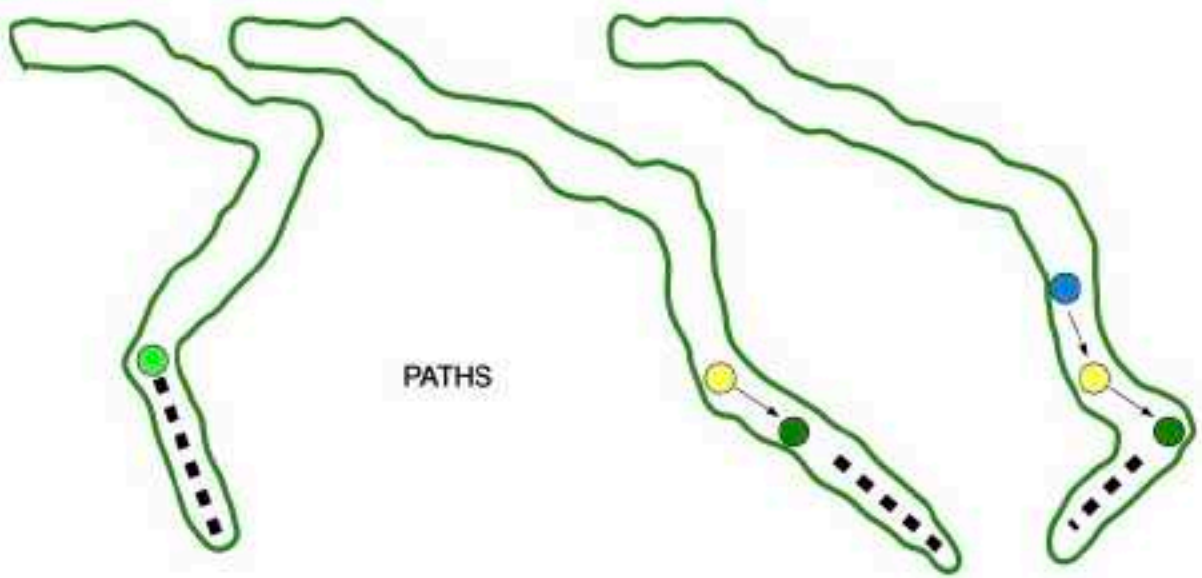
Collect the states (Reachable States)

Compute the states through which the system transits: (NOT ALWAYS POSSIBLE!!!!!)



Collect the states (Reachable States)

Compute the states through which the system transits: (NOT ALWAYS POSSIBLE!!!!!)

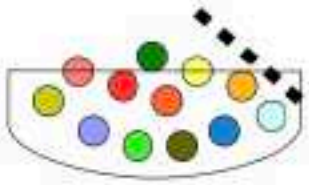


STATES

Collect the states (Reachable States)

Compute the states through which the system transits: (NOT ALWAYS POSSIBLE!!!!!)

PATHS



STATES

Formal Methods

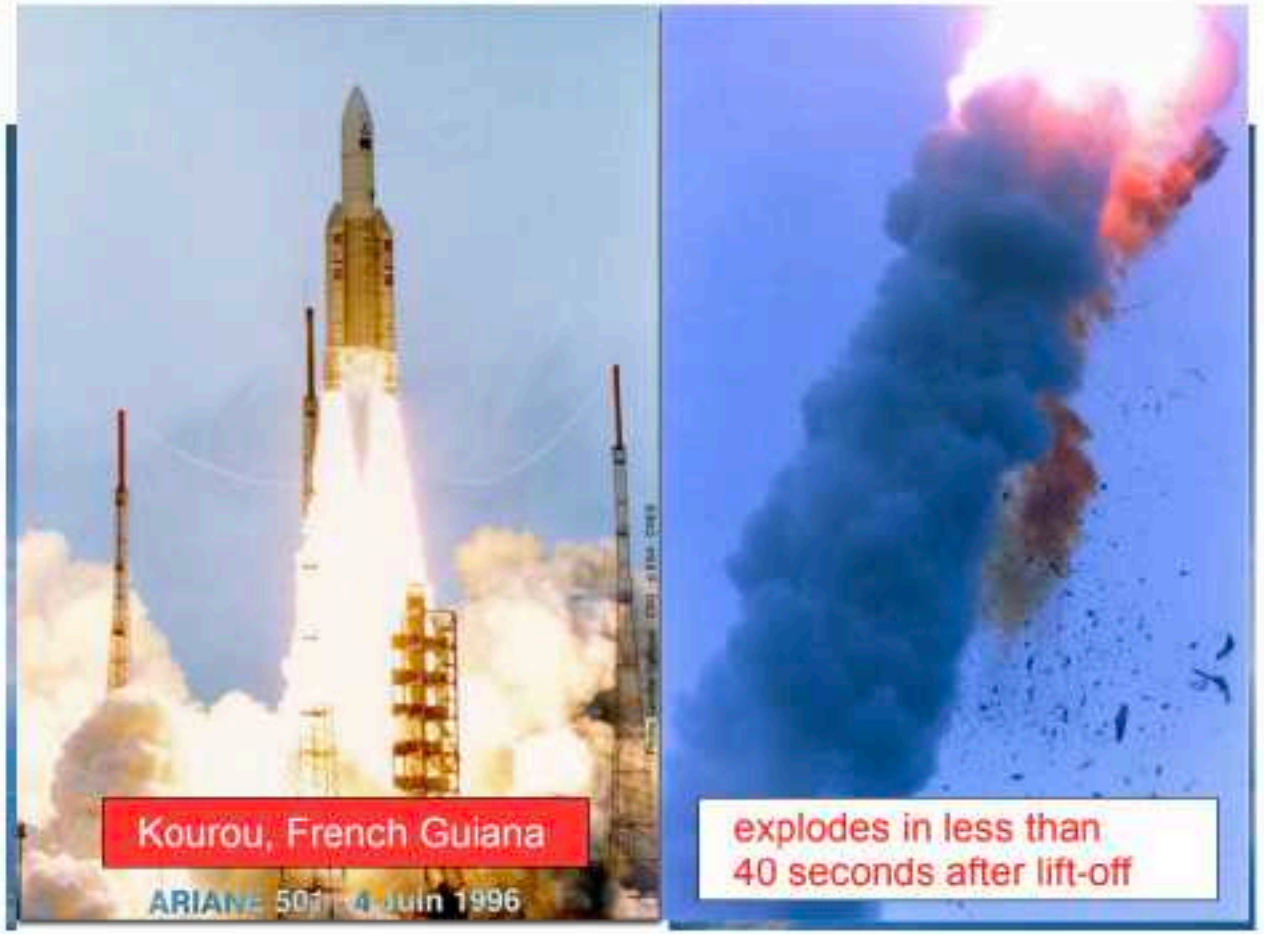
If the trade is so simple, what is the challenge!!!!!

- Systems are continuous: infinite stated and
- non-terminating: infinite runs
- requirements: temporal

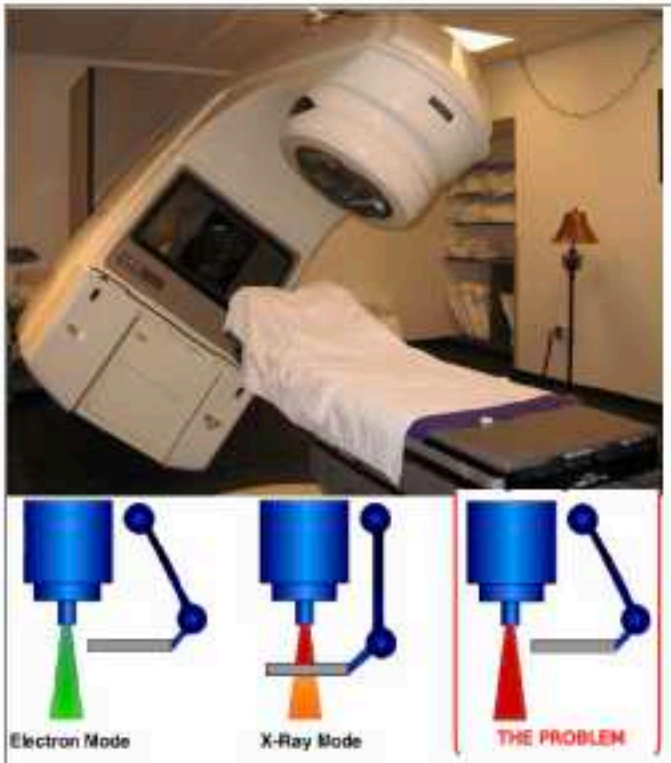
Undecidability with decision procedures!!!!!

Software reliability in Safety Critical Systems

Software reliability in Safety Critical Systems



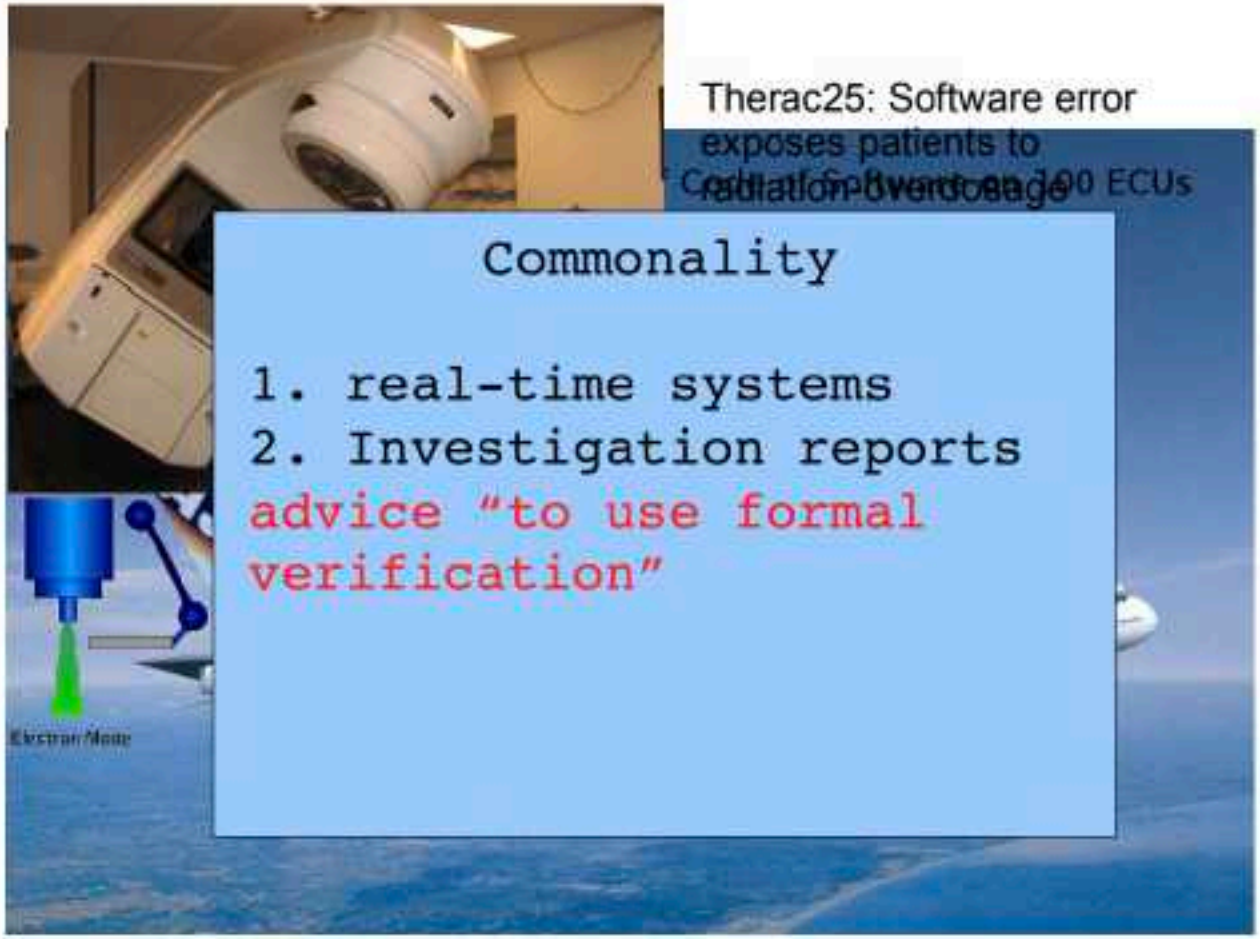
Software reliability in Safety Critical Systems



Therac25: Software error exposes patients to radiation-overdosage

Atleast 5 people died

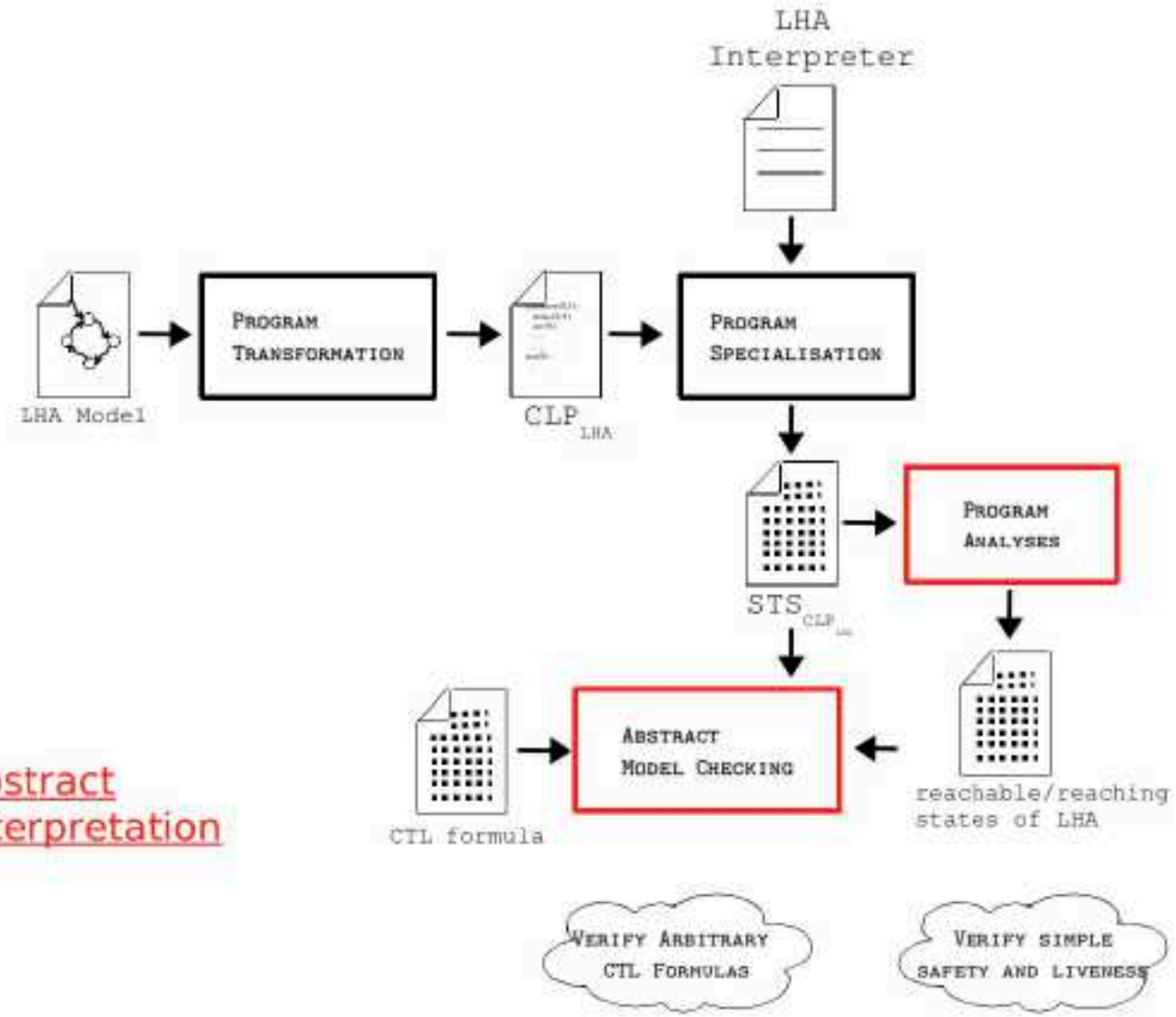
Software reliability in Safety Critical Systems



Software reliability in Safety Critical Systems

Toyota Prius NASA-Investigation

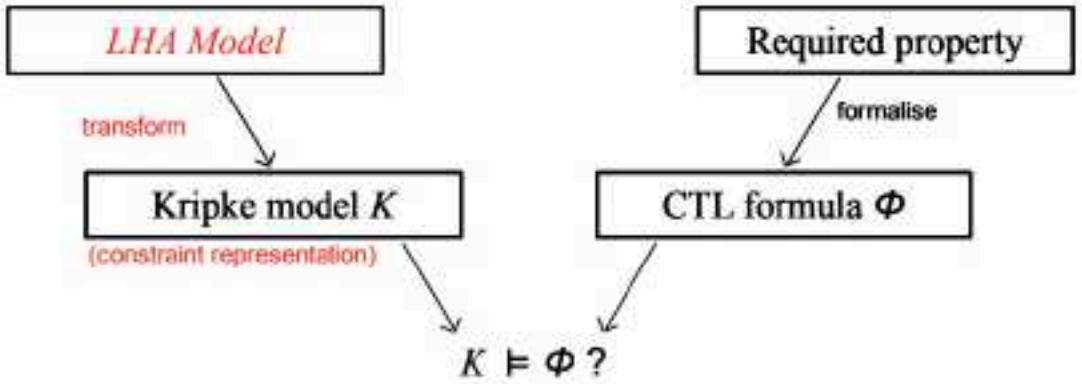
Proposed Analysis framework



Abstract Interpretation

Model checking

- due to *Ed Clarke et. al*: Turing Award Winners (joint)
- algorithmic technique to verify temporal properties of transition systems



- $K \models \phi$ i.e. $\forall s \in \texttt{InitStates} : s, K \models \phi$
  1. Compute $[\![\phi]\!]$, the set of states where ϕ holds;
  2. Check $\texttt{InitStates} \subseteq [\![\phi]\!]$.